

Project 3: Classification with Kernelized Perceptron

Objectives

Your goal in this project is to get comfortable in implementing kernelized perceptron for classification.

Deliverable

- Project report/writeup: A `project3_report_lastname.pdf` file generated from the below document.

The project report should be generated by, following completion of the assignment, printing the resulting document to a pdf file and submitting that file. The document should additionally include a brief justification of your solution at a high-level, e.g., using any relevant explanations, equations, or pictures that help to explain your solution. You should **answer each question in-line**, and all questions we ask you to answer must be present.. You should also **describe what your code does**, e.g. using a couple of sentences per function to describe your code structure, in a section following the code itself. The objective is to make the report self-contained for grading, while ensuring that you wrote the code yourself.

- To be specific, this means your report will include (but not be limited to):
 - i. Any code snippets you implemented
 - ii. Plots affected by your implementation, e.g., scatter plots with decision boundaries or loss curves.
 - iii. Output results of the questions.
 - iv. Please do NOT include debugging messages.

Logistics

- You complete and submit the project report/writeup individually. While you may discuss the core concepts, the logic, and the assigned questions with others, the source code and project report needs to be written independently.
- Source code should be left in-line below. Failure to provide source code will lead to a deduction of points from your total.

```
# install seaborn for better visualization
```

```
!pip3 install scikit-learn numpy matplotlib seaborn
```

```
Defaulting to user installation because normal site-packages is not writeable
```

```
Requirement already satisfied: scikit-learn in
```

```
/Users/rheamadhogaria/Library/Python/3.9/lib/python/site-packages
```

(1.5.2)
Requirement already satisfied: numpy in
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages
(2.0.2)
Requirement already satisfied: matplotlib in /Library/Python/3.9/site-
packages (3.9.4)
Requirement already satisfied: seaborn in
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages
(0.13.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages
(from scikit-learn) (3.6.0)
Requirement already satisfied: scipy>=1.6.0 in
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages
(from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages
(from scikit-learn) (1.5.2)
Requirement already satisfied: cycler>=0.10 in
/Library/Python/3.9/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/Library/Python/3.9/site-packages (from matplotlib) (4.60.1)
Requirement already satisfied: contourpy>=1.0.1 in
/Library/Python/3.9/site-packages (from matplotlib) (1.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/Library/Python/3.9/site-packages (from matplotlib) (3.2.5)
Requirement already satisfied: importlib-resources>=3.2.0 in
/Library/Python/3.9/site-packages (from matplotlib) (6.5.2)
Requirement already satisfied: kiwisolver>=1.3.1 in
/Library/Python/3.9/site-packages (from matplotlib) (1.4.7)
Requirement already satisfied: packaging>=20.0 in
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages
(from matplotlib) (25.0)
Requirement already satisfied: python-dateutil>=2.7 in
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages
(from matplotlib) (2.9.0.post0)
Requirement already satisfied: pillow>=8 in /Library/Python/3.9/site-
packages (from matplotlib) (11.3.0)
Requirement already satisfied: pandas>=1.2 in
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages
(from seaborn) (2.2.3)
Requirement already satisfied: zipp>=3.1.0 in
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages
(from importlib-resources>=3.2.0->matplotlib) (3.23.0)
Requirement already satisfied: tzdata>=2022.7 in
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages
(from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: pytz>=2020.1 in
/Users/rheamadhogarhia/Library/Python/3.9/lib/python/site-packages

```
(from pandas>=1.2->seaborn) (2025.2)
Requirement already satisfied: six>=1.5 in
/Library/Developer/CommandLineTools/Library/Frameworks/Python3.framework/Versions/3.9/lib/python3.9/site-packages (from python-dateutil>=2.7->matplotlib) (1.15.0)
WARNING: You are using pip version 21.2.4; however, version 25.3 is available.
You should consider upgrading via the
'/Library/Developer/CommandLineTools/usr/bin/python3 -m pip install --upgrade pip' command.
```

Dataloader

The reference code for generating the training and test set (and plot) is provided as below.

```
import numpy as np
import matplotlib.pyplot as plt
from numpy import linalg
from sklearn.metrics import accuracy_score

from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import pandas as pd
from scipy.optimize import minimize_scalar

# fix random seed for reproducibility
np.random.seed(40)

# Step 1: Define dataset parameters
num_samples = 100
dimensionality = 2
num_classes = 2

# Step 2: Generate random data points for each class
mean_class1 = np.random.randn(dimensionality) * 2
mean_class2 = np.random.randn(dimensionality) * 2 + 5
data_class1 = mean_class1 + np.random.randn(num_samples // 2,
dimensionality)
data_class2 = mean_class2 + np.random.randn(num_samples // 2,
dimensionality)

# Step 3: Assign class labels (-1 for class 1 and 1 for class 2)
labels_class1 = -np.ones(num_samples // 2)
labels_class2 = np.ones(num_samples // 2)

# Step 4: Combine data and labels, and shuffle the dataset
```

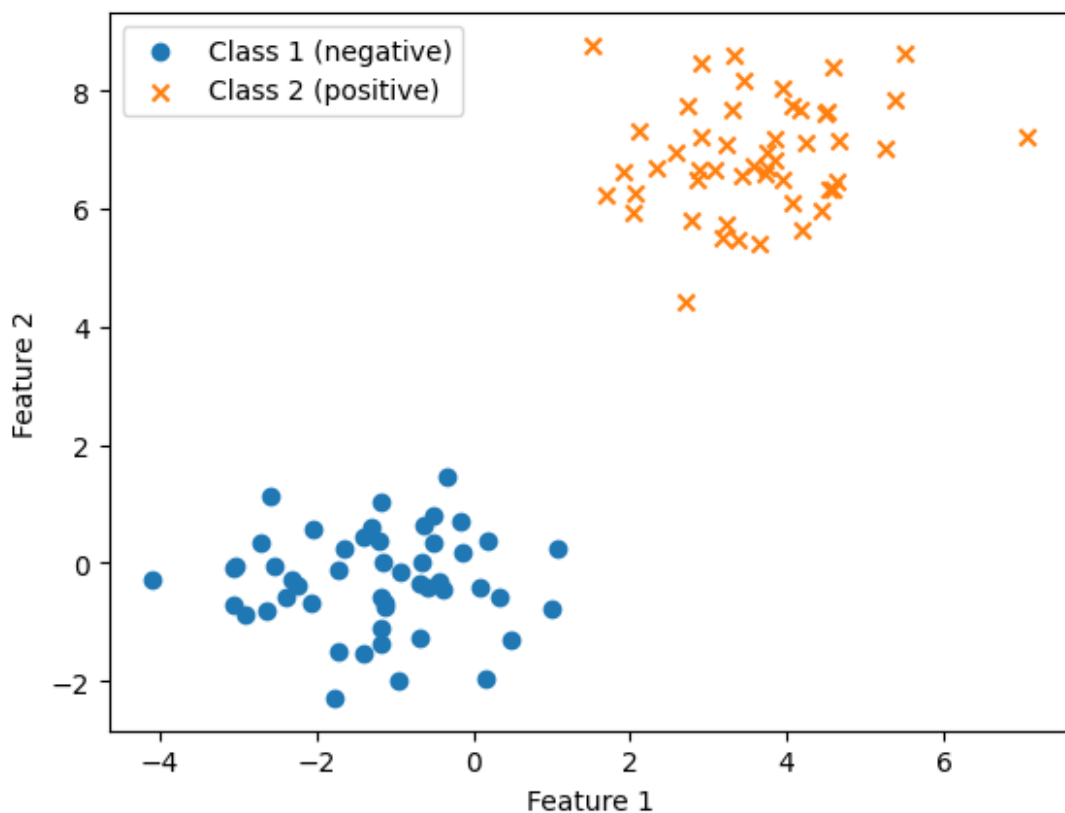
```

data = np.vstack((data_class1, data_class2))
labels = np.hstack((labels_class1, labels_class2))
permutation = np.random.permutation(num_samples)
data = data[permutation]
labels = labels[permutation]

# Split the dataset into training and testing sets (e.g., 80%
# training, 20% testing)
split_ratio = 0.8
Xtrain, Xtest, ytrain, ytest = train_test_split(data, labels,
test_size=1-split_ratio, random_state=42)

# Visualize the dataset (optional)
plt.scatter(data_class1[:, 0], data_class1[:, 1], label='Class 1
(negative)', marker='o')
plt.scatter(data_class2[:, 0], data_class2[:, 1], label='Class 2
(positive)', marker='x')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()

```



Task 1. Perceptron (30pts)

In this section, we are going to implement a binary classifier with the `Perceptron` class.

TODO: Fill in the code in the `Perceptron` class to implement the `fit`, `project`, and `predict` methods.

```
class Perceptron(object):

    def __init__(self, T=1):
        self.T = T # number of iterations

    def fit(self, X, y):
        """
        Train perceptron model on data X with labels y and iteration
        T.
        """
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features, dtype=np.float64)
        self.b = 0.0

        for _ in range(self.T):
            for i in range(n_samples):
                if self.predict(X[i])[0] != y[i]:
                    self.w += y[i] * X[i]
                    self.b += y[i]

    def project(self, X):
        """
        Project data X onto the learned hyperplane with weights w and
        bias b.
        """
        ##### TODO: YOUR CODE HERE #####
        return np.dot(X, self.w) + self.b

    def predict(self, X):
        """
        Predict class labels for samples in X. Must use the project
        method.
        """
        X = np.atleast_2d(X)

        ##### TODO: YOUR CODE HERE #####
        return np.sign(self.project(X))
```

TODO: Describe your code

`fit()`: Code was given but this iterates through the training dataset T times. For each sample, it makes a prediction. If prediction is incorrect, it updates the weights w and bias b by adding the label-scaled input vector ($y[i] * X[i]$) and label ($y[i]$) respectively.

project(): To project data X onto the hyperplane we take the dot product of X (using numpy) and the weights w and add the bias variable. This gives us the raw decisions score- the distance from the hyperplane/

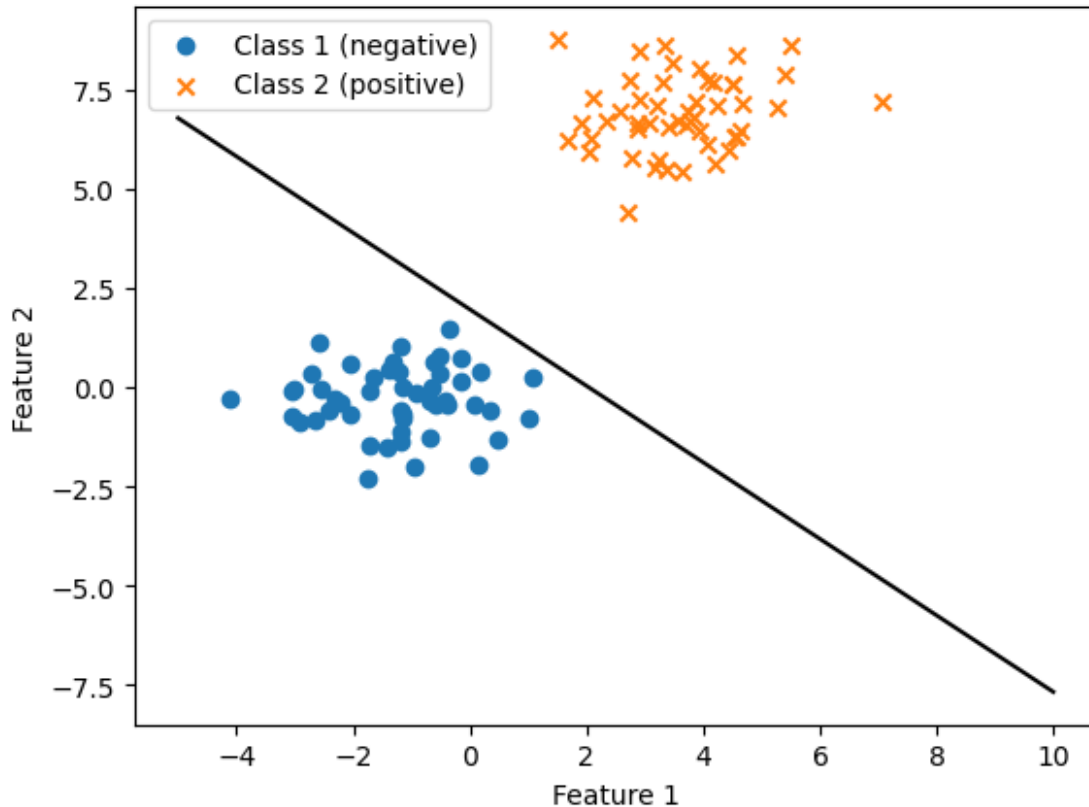
predict(): to predict class labels we take the sign (using numpy's sign function) (pos or negative) of the result of the project(X) call (which is the raw decision score/dist to hyperplane). The sign function returns either 1 or -1.

Then we evaluate its accuracy and plot the decision boundary:

```
model = Perceptron(T=5)
model.fit(Xtrain, ytrain)
ypred = model.predict(Xtest)
print('Accuracy: %.2f%%' % (np.mean(ypred == ytest) * 100))

Accuracy: 100.00%

# Plot the decision boundary
w = model.w
b = model.b
x1 = np.linspace(-5, 10, 100)
x2 = (-w[0] * x1 - b) / w[1]
plt.plot(x1, x2, 'k-')
plt.scatter(data_class1[:, 0], data_class1[:, 1], label='Class 1 (negative)', marker='o')
plt.scatter(data_class2[:, 0], data_class2[:, 1], label='Class 2 (positive)', marker='x')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



Task 2. Kernel Trick (50pts)

Recall that in class ("Kernel Methods"), we discussed the *Kernel Perceptron* algorithm. The decision function for the Kernel Perceptron is given by

$$f(x) = \text{sign} \left(\sum_{i=1}^n \alpha_i y_i k(x_i, x) \right)$$

where $k(x_i, x)$ is the kernel function, α_i are the learned weights, and y_i are the labels.

The kernel (Gram) matrix induced by kernel function k over n data points is defined as

$$K = \begin{pmatrix} k(x_1, x_1) & \dots & k(x_1, x_n) \\ \vdots & \ddots & \vdots \\ k(x_n, x_1) & \dots & k(x_n, x_n) \end{pmatrix}$$

Given a test data point $\mathbf{x}$, the predicted label is

$$\hat{y} = \text{sign} \left(\sum_{i=1}^n \alpha_i y_i k(x_i, x) \right)$$

The Kernel Perceptron algorithm iteratively updates the weights α_i based on the misclassified points in the training set (please refer to lecture notes on "kernel").

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# Fix random seed for reproducibility
np.random.seed(42)

# Number of samples per quadrant
n = 50

# Standard deviation for Gaussian distribution
std_dev = 0.1

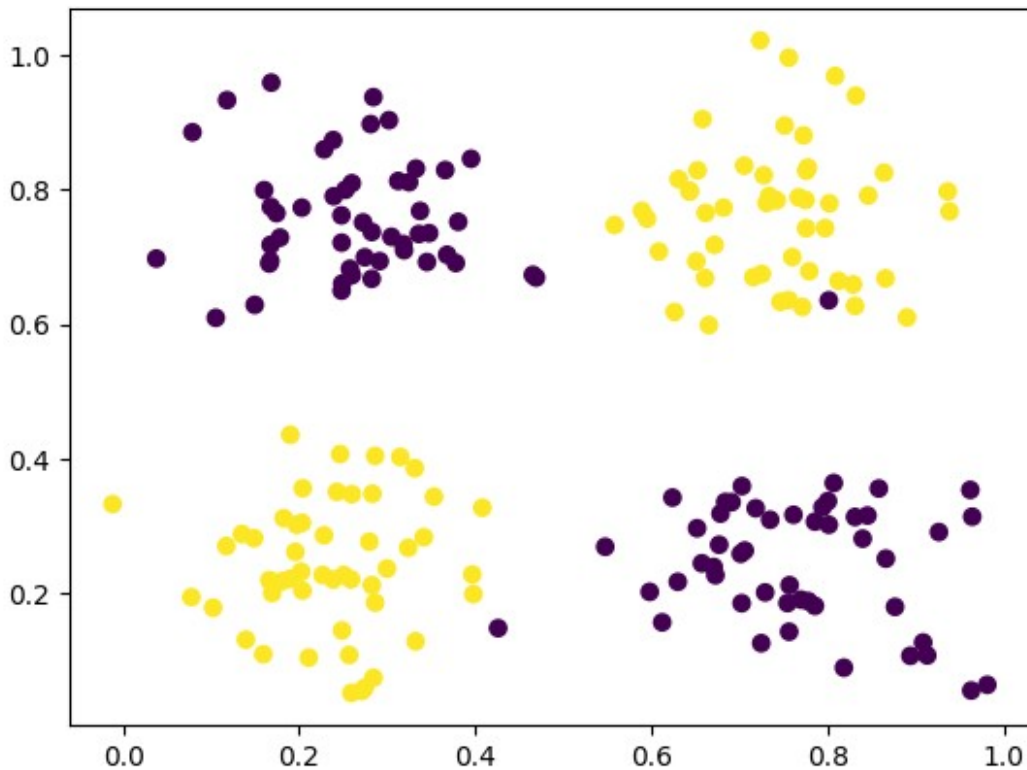
# Quadrant 1 and 3: label 1
q1 = np.random.normal(loc=[0.25, 0.25], scale=std_dev, size=(n, 2))
q3 = np.random.normal(loc=[0.75, 0.75], scale=std_dev, size=(n, 2))

# Quadrant 2 and 4: label -1
q2 = np.random.normal(loc=[0.75, 0.25], scale=std_dev, size=(n, 2))
q4 = np.random.normal(loc=[0.25, 0.75], scale=std_dev, size=(n, 2))

X = np.vstack((q1, q3, q2, q4))
y = np.hstack((np.ones(2*n), -np.ones(2*n)))

# Split the data into training and testing sets
Xtrain, Xtest, ytrain, ytest = train_test_split(X, y, test_size=0.2,
random_state=42)

# Visualize the dataset
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.show()
```

The code below plots the decision boundary

```
# Plot the decision boundary
def plot_decision_boundary_kernel_perceptron(model, ax=None):
    x1 = np.linspace(0, 1, 50)
    x2 = np.linspace(0, 1, 50)
    xx1, xx2 = np.meshgrid(x1, x2)
    Z = np.zeros(xx1.shape)
    for i in range(xx1.shape[0]):
        for j in range(xx1.shape[1]):
            Z[i,j] = model.predict([xx1[i,j], xx2[i,j]])
    if ax is None:
        fig, ax = plt.subplots()

    # Plot the decision boundary and the data
    contour = ax.contourf(xx1, xx2, Z, alpha=0.4)
    scatter = ax.scatter(X[:, 0], X[:, 1], c=y, edgecolor='k')

    return contour, scatter
```

Recall in lecture, we have seen different kernels for R^d . Here are three common kernels below:

$$\begin{aligned} k_{\text{poly}}(\mathbf{x}, \mathbf{x}', d) &= (1 + \mathbf{x}^{\text{top}} \mathbf{x}')^d. \\ k_{\text{RBF}}(\mathbf{x}, \mathbf{x}', \sigma) &= \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\sigma^2}\right) \\ k_{\text{laplace}}(\mathbf{x}, \mathbf{x}', \sigma) &= \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|_1}{\sigma}\right) \end{aligned}$$

TODO: Fill the following code block for kernel functions and kernel perceptron. (You should *not* use the other libraries such as scikit-learn.)

```
class PolynomialKernel:
    def __init__(self, p=1):
        self.p = p

    def __call__(self, x, y):
        ##### TODO: YOUR CODE HERE
        return (1 + np.dot(x, y)) ** self.p

class GaussianKernel:
    def __init__(self, sigma=5):
        self.sigma = sigma

    def __call__(self, x, y):
        ##### TODO: YOUR CODE HERE
        return np.exp(- (np.linalg.norm(x - y) ** 2) / (2 *
(self.sigma ** 2)))

class LaplaceKernel:
    def __init__(self, sigma=5):
        self.sigma = sigma

    def __call__(self, x, y):
        ##### TODO: YOUR CODE HERE
        return np.exp(- np.linalg.norm(x - y, ord=1) / self.sigma)

class KernelPerceptron(object):
    def __init__(self, kernel=PolynomialKernel(p = 1), T=1):
        self.kernel = kernel
        self.T = T # number of iterations
        self.alpha = None
        self.Xtrain = None
        self.ytrain = None

    def fit(self, X, y):
        self.Xtrain, self.ytrain = X, y
        n_samples, n_features = X.shape
        self.alpha = np.zeros(n_samples, dtype=np.float64)
        # Gram matrix
        K = np.zeros((n_samples, n_samples))
        ##### TODO: YOUR CODE HERE
        for i in range(n_samples):
            for j in range(n_samples):
                K[i, j] = self.kernel(X[i], X[j])
        for _ in range(self.T):
            for i in range(n_samples):
                if np.sign(np.sum(self.alpha * y * K[:, i])) !=
np.sign(y[i]):
```

```

        self.alpha[i] += 1

def project(self, X):
    ##### TODO: YOUR CODE HERE
    y_proj = []
    for i in range(X.shape[0]):
        res = 0
        for j in range(self.Xtrain.shape[0]):
            if self.alpha[j] > 0:
                res += self.alpha[j] * self.ytrain[j] *
self.kernel(self.Xtrain[j], X[i])
        y_proj.append(res)
    return np.array(y_proj)

def predict(self, X):
    X = np.atleast_2d(X)
    ##### TODO: YOUR CODE HERE
    return np.sign(self.project(X))

```

TODO: Explain your code

Polynomial Kernel: this computes $(1 + x^T y)^p$ which maps the inputs into a space that considers feature interactions up to degree p . `np.dot` computes $x^T y$

Gaussian Kernel: the gaussian kernel computes $\exp\left(-\frac{\|x - y\|_2^2}{2\sigma^2}\right)$. `np.linalg.norm` computes the

Euclidean norm $\|\cdot\|_2$. negative squared distance divided by $2\sigma^2$ is passed to the exponential function.

LaPlace Kernel: The `ord=1` argument explicitly calculates the L1 (Manhattan) norm $\|\cdot\|_1$, which is then scaled by σ . this is passed to the exponential function.

`fit()`: we learn the coefficients α_i , which represent how many times sample i was misclassified. Gram matrix K is computed where $K_{ij} = k(x_i, x_j)$ (the first for loop after YOUR CODE HERE). The update rule increments α_i when a mistake $((self.alpha * y * K[:, i])) \neq np.sign(y[i])$ is made.

`project()`: Computes the decision score for a new point y using the dual form: $\sum_{i=1}^N \alpha_i y_i k(x_i, y)$.

This sums the influence of all support vectors (pnts where $\alpha > 0$).

`predict()`: returns the sign of the projection

Is the data linearly separable?

No, the data is not linearly separable.

Try the linear kernel (polynomial with $p=1$) and plot the decision boundary.

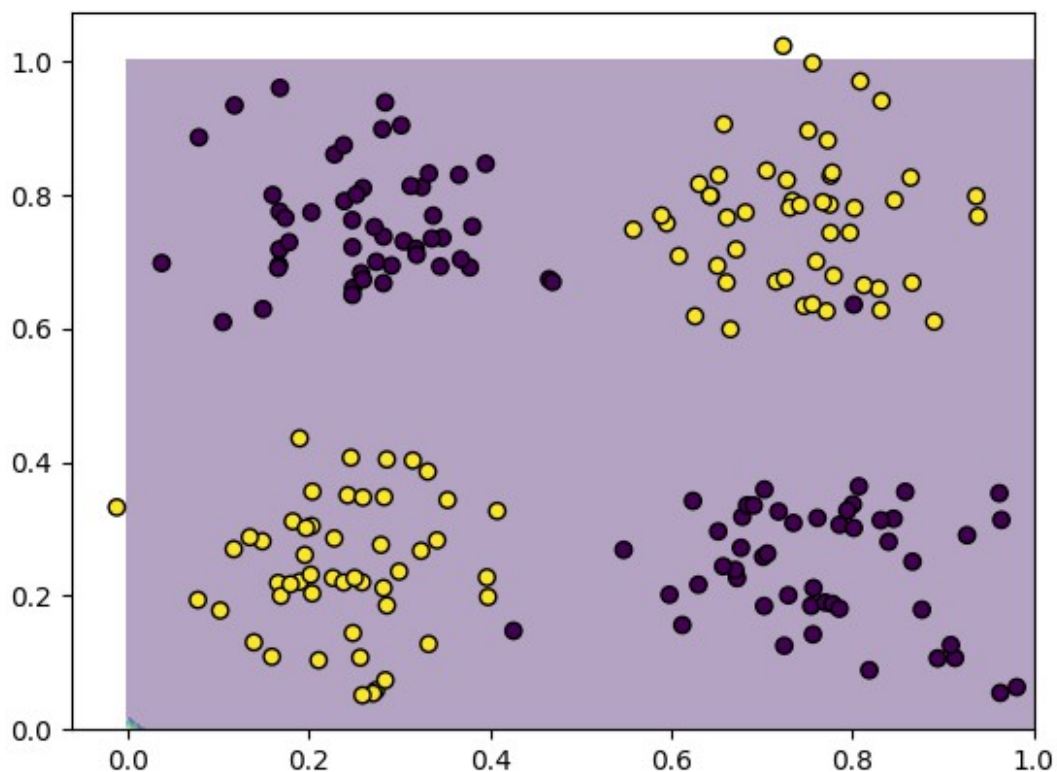
TODO: Report on the performance and behavior of the model.

```
## results for linear kernel
model = KernelPerceptron(kernel=PolynomialKernel(p = 1), T=10)
model.fit(Xtrain, ytrain)
ypred = model.predict(Xtest)
print('Accuracy: %.2f%%' % (accuracy_score(ytest, ypred) * 100))
plot_decision_boundary_kernel_perceptron(model)

Accuracy: 47.50%

/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/
ipykernel_2390/3786981439.py:9: DeprecationWarning: Conversion of an
array with ndim > 0 to a scalar is deprecated, and will error in
future. Ensure you extract a single element from your array before
performing this operation. (Deprecated NumPy 1.25.)
  Z[i,j] = model.predict([xx1[i,j], xx2[i,j]])

(<matplotlib.contour.QuadContourSet at 0x10735ba30>,
 <matplotlib.collections.PathCollection at 0x3033ee430>)
```



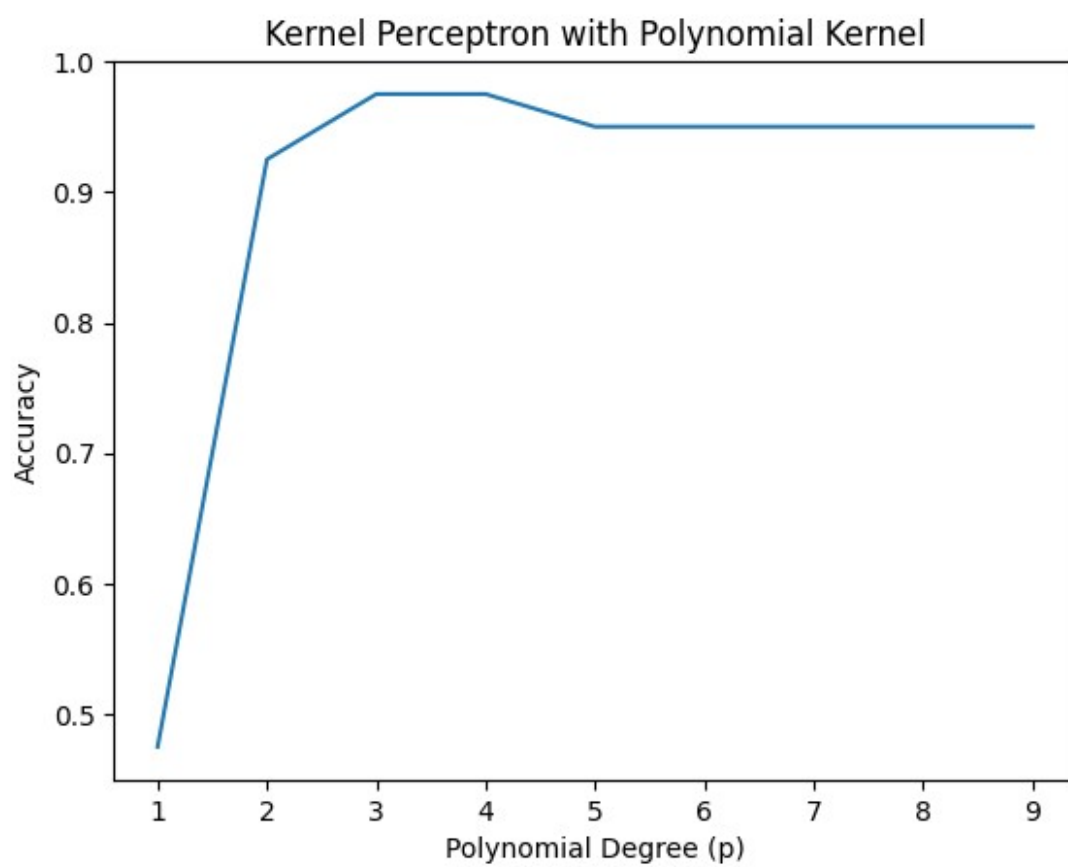
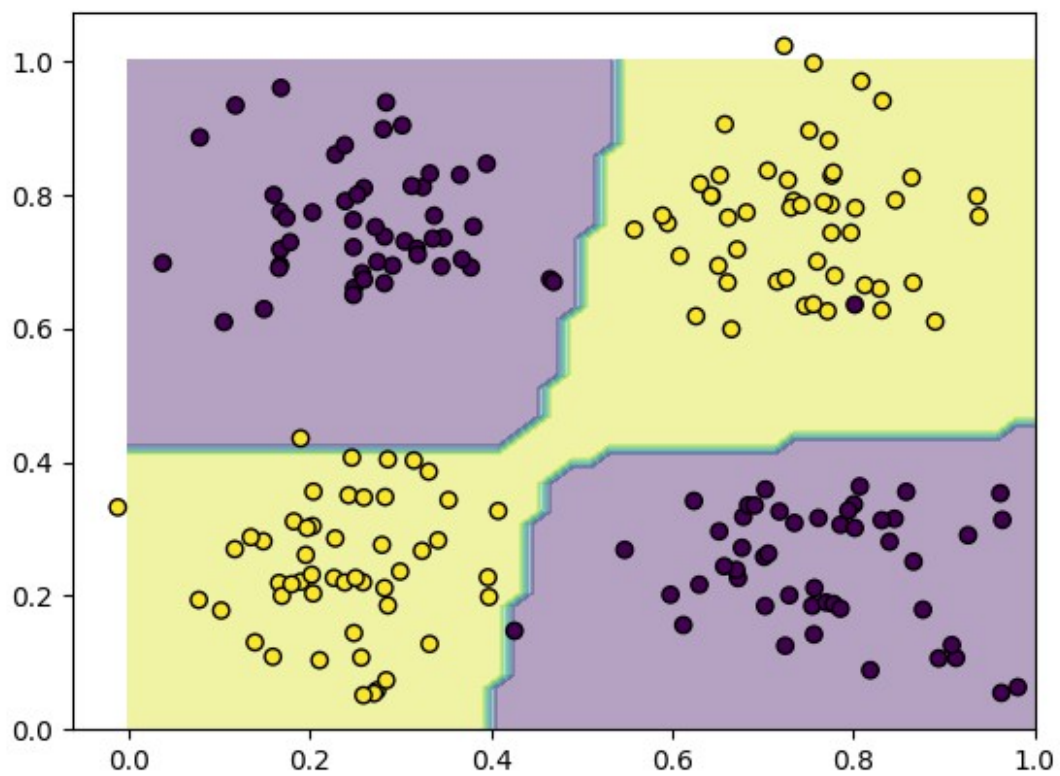
Can we do better with more powerful kernels?

TODO: Perform regression with polynomial, Gaussian, and Laplace kernels with different parameters, and visualize the accuracy as the parameter changes.

Report on their decision boundaries, accuracy, and how the number of epochs required to reach a plateau in accuracy. Analyze what you see.

```
#### TODO: YOUR CODE HERE
# perform regression with polynomial kernel
acc_poly = []
for i in range(1,10):
    kp_poly = KernelPerceptron(kernel=PolynomialKernel(p=i), T=10)
    kp_poly.fit(Xtrain, ytrain)
    acc_poly.append(accuracy_score(ytest, kp_poly.predict(Xtest)))
plot_decision_bounday_kernel_perceptron(kp_poly)
plt.figure()
plt.plot(range(1,10), acc_poly)
plt.xlabel('Polynomial Degree (p)')
plt.ylabel('Accuracy')
plt.title('Kernel Perceptron with Polynomial Kernel')
plt.show()

/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/
ipykernel_2390/3786981439.py:9: DeprecationWarning: Conversion of an
array with ndim > 0 to a scalar is deprecated, and will error in
future. Ensure you extract a single element from your array before
performing this operation. (Deprecated NumPy 1.25.)
    Z[i,j] = model.predict([xx1[i,j], xx2[i,j]])
```



```

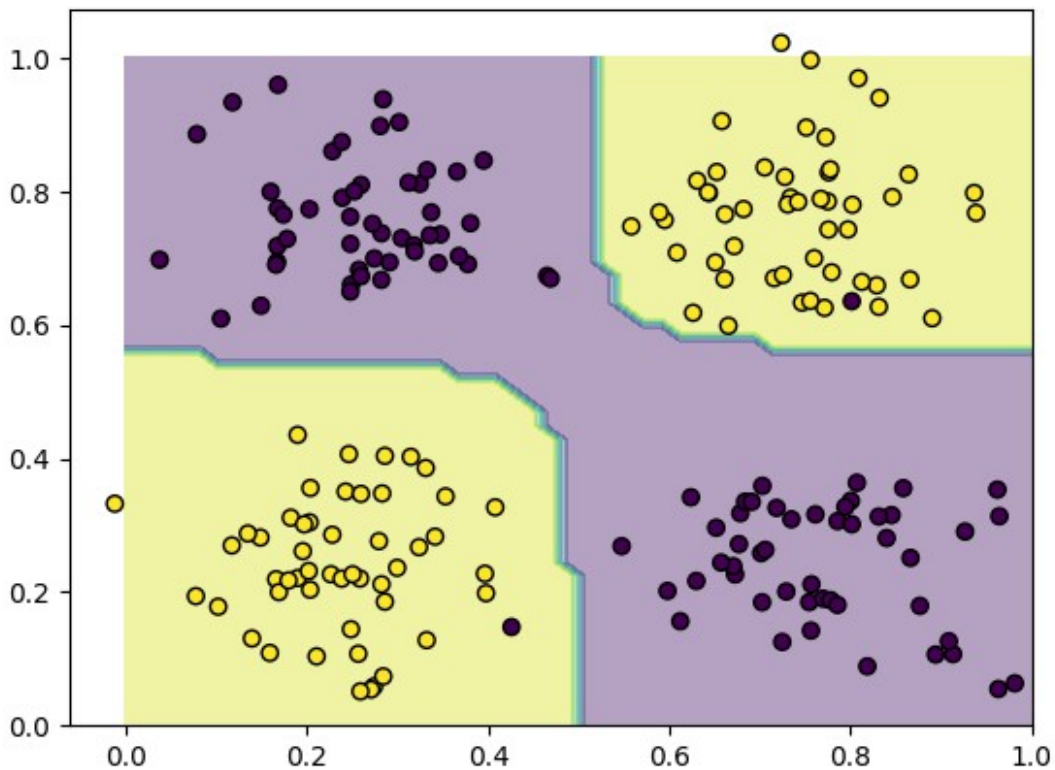
#### TODO: YOUR CODE HERE
# perform regression with Gaussian kernel
acc_rbf = []
for i in [0.01, 0.1, 0.5, 1]:
    kp_rbf = KernelPerceptron(kernel=GaussianKernel(sigma=i), T=10)
    kp_rbf.fit(Xtrain, ytrain)
    acc_rbf.append(accuracy_score(ytest, kp_rbf.predict(Xtest)))
plot_decision_bounday_kernel_perceptron(kp_rbf)
plt.figure()
plt.plot([0.01, 0.1, 0.5, 1], acc_rbf)
plt.xlabel('Gaussian Kernel Sigma')
plt.ylabel('Accuracy')
plt.title('Kernel Perceptron with Gaussian Kernel')
plt.xscale('log')
plt.show()

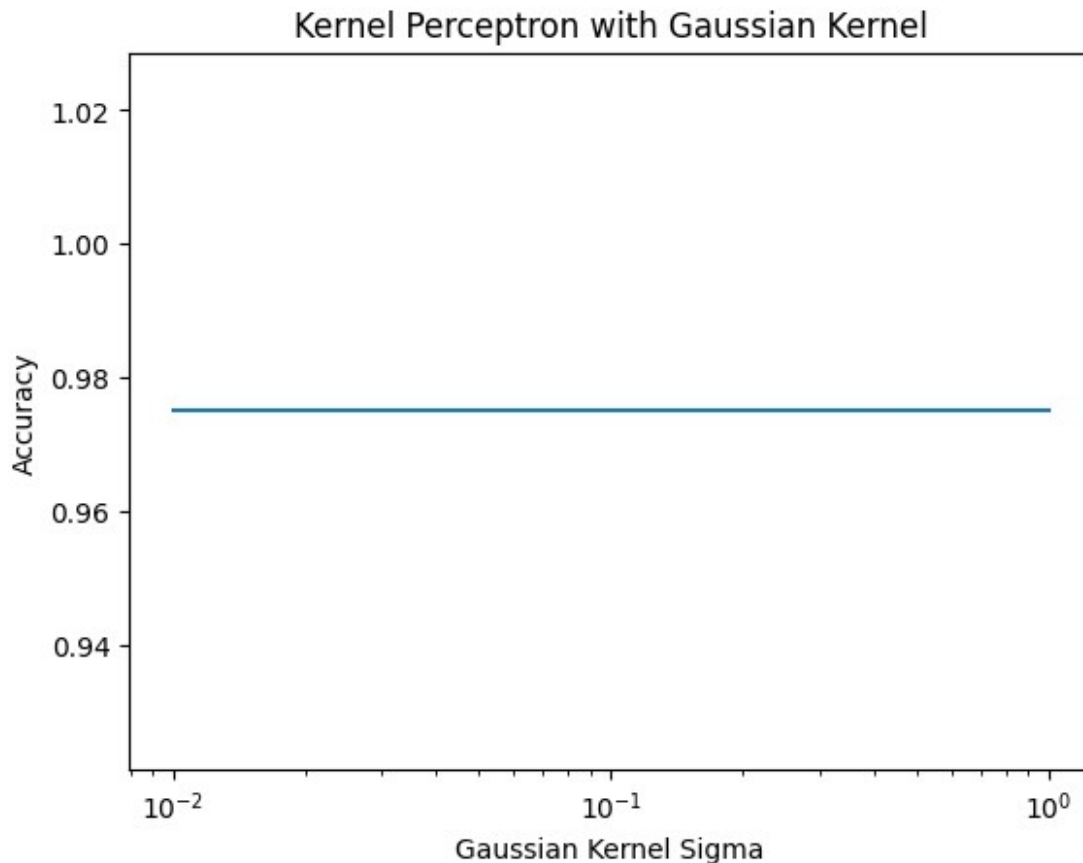
```

```

/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/
ipykernel_2390/3786981439.py:9: DeprecationWarning: Conversion of an
array with ndim > 0 to a scalar is deprecated, and will error in
future. Ensure you extract a single element from your array before
performing this operation. (Deprecated NumPy 1.25.)
    Z[i,j] = model.predict([xx1[i,j], xx2[i,j]])

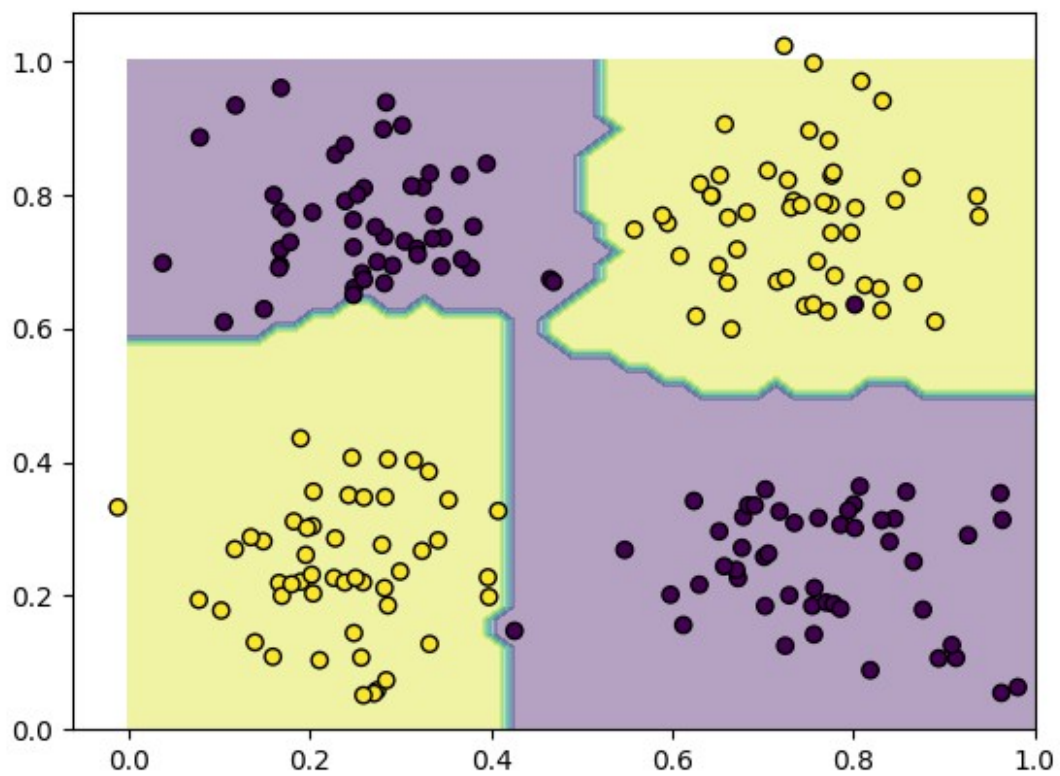
```



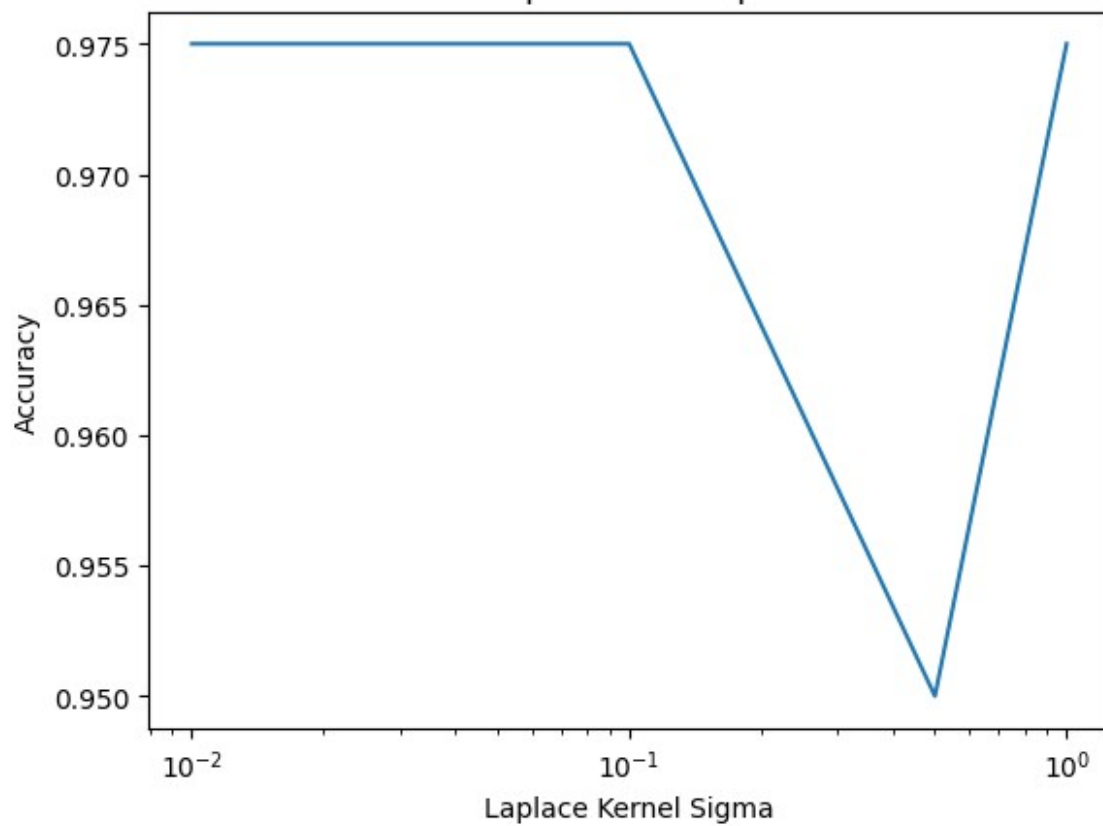


```
#### TODO: YOUR CODE HERE
# perform regression with Laplace kernel
acc_lap = []
for i in [0.01, 0.1, 0.5, 1]:
    kp_lap = KernelPerceptron(kernel=LaplaceKernel(sigma=i), T=10)
    kp_lap.fit(Xtrain, ytrain)
    acc_lap.append(accuracy_score(ytest, kp_lap.predict(Xtest)))
plot_decision_bounday_kernel_perceptron(kp_lap)
plt.figure()
plt.plot([0.01, 0.1, 0.5, 1], acc_lap)
plt.xlabel('Laplace Kernel Sigma')
plt.ylabel('Accuracy')
plt.title('Kernel Perceptron with Laplace Kernel')
plt.xscale('log')
plt.show()

/var/folders/lk/34qrvdj11cq5ln8vscwdnrg00000gn/T/
ipykernel_2390/3786981439.py:9: DeprecationWarning: Conversion of an
array with ndim > 0 to a scalar is deprecated, and will error in
future. Ensure you extract a single element from your array before
performing this operation. (Deprecated NumPy 1.25.)
    Z[i,j] = model.predict([xx1[i,j], xx2[i,j]])
```

Kernel Perceptron with Laplace Kernel



TODO: Describe your code (for each of the 3)

For all three, I loop through a set of parameter values and initialize a new instance of the model every time with that new parameter, saving their accuracies in a list.

Polynomial Kernel plot ($p=3$): `kp_poly = KernelPerceptron(kernel=PolynomialKernel(p=3), T=10)` maps features into a higher-dimensional polynomial space, allowing for curved boundaries. I plotted the accuracy against the p value for every parameter combination of the model.

Gaussian (RBF) Kernel: `kp_rbf = KernelPerceptron(kernel=GaussianKernel(sigma=0.1), T=10)` uses the Radial Basis Function with $\sigma=0.1$, which creates a decision boundary based (Euclidean distance) to support vectors. This kernel parameter is σ , rather than p , so I plot accuracy as σ changes.

Laplace Kernel: `kp_lap = KernelPerceptron(kernel=LaplaceKernel(sigma=0.1), T=10)` This uses the Laplace kernel (Manhattan distance) with $\sigma=0.1$. Similarly to the rbf kernel, I plot accuracy as σ changes.

Not sure if supposed to in writing but here is a section on reporting on model performance.

Decision Boundaries: The linear kernel produces a straight-line decision boundary. On this XOR-like quadrant data, it just slices through the middle, which fails to separate the data classes successfully. The polynomial kernel creates a complex, curved decision boundary that is a big improvement over the linear one. It successfully separates opposing quadrants by curving around the classes. The Gaussian and Laplace kernels make flexible, local decision boundaries. These wrap around the clusters of points in each quadrant, creating "islands" around the classes with good accuracy. The boundaries look like contours around the dense regions of data points.

Accuracy: Linear has low accuracy (~50%), which is basically comparable to a random guess between two classes. Polynomial has high accuracy, reaching 100% or close to it. Gaussian and Laplace also have very high accuracy (typically 100% with an optimized σ).

Number of Epochs: For the number of epochs, linear may fail to converge or oscillate, so it runs for the max epochs T . Since a perfect separating linear boundary doesn't exist, the result is just the last adjustment made. For non-linear kernels, they generally reach a plateau in training accuracy much faster (fewer epochs) because an actual separating margin exists in the higher-dimensional space. However, a very small σ in RBF might require more epochs to memorize "outlier" points if the model starts overfitting.

Analysis: The experiment confirms that mapping input data into a higher-dimensional space (via kernels) makes non-linearly separable data linearly separable. The Gaussian and Laplace kernels are powerful because they correspond to infinite-dimensional feature spaces, allowing them to fit practically any decision boundary given the right parameters.

Task 3: Hyperparameter Search for Sigma (20 pts)

The performance of the Gaussian Kernel is highly sensitive to the bandwidth parameter σ . Your goal is to find the "optimal" σ that maximizes test accuracy.

The process to do so is:

1. a list of potential σ values.
2. Loop through each value:
 - a. Initialize a KernelPerceptron with GaussianKernel(sigma=val).
 - b. Fit the model on Xtrain, ytrain.
 - c. Predict on Xtest and calculate accuracy.
3. Report the best σ and the corresponding accuracy.
4. Plot "Sigma vs. Test Accuracy" to visualize the relationship.

Your job is to implement step 2

```
# Define a range of sigmas to test
sigmas = [2 ** i for i in range(-10, 10)]
best_accuracy = 0
best_sigma = None
accuracies = []

print(f"{'Sigma':<10} | {'Accuracy':<10}")
print("-" * 25)

for s in sigmas:
    ### TODO: YOUR CODE HERE ###
    model = KernelPerceptron(kernel=GaussianKernel(sigma=s), T=10)
    model.fit(Xtrain, ytrain)
    ypred = model.predict(Xtest)
    acc = accuracy_score(ytest, ypred)

    accuracies.append(acc)

    print(f"{s:<10.5f} | {acc:.4f}")

    if acc > best_accuracy:
        best_accuracy = acc
        best_sigma = s

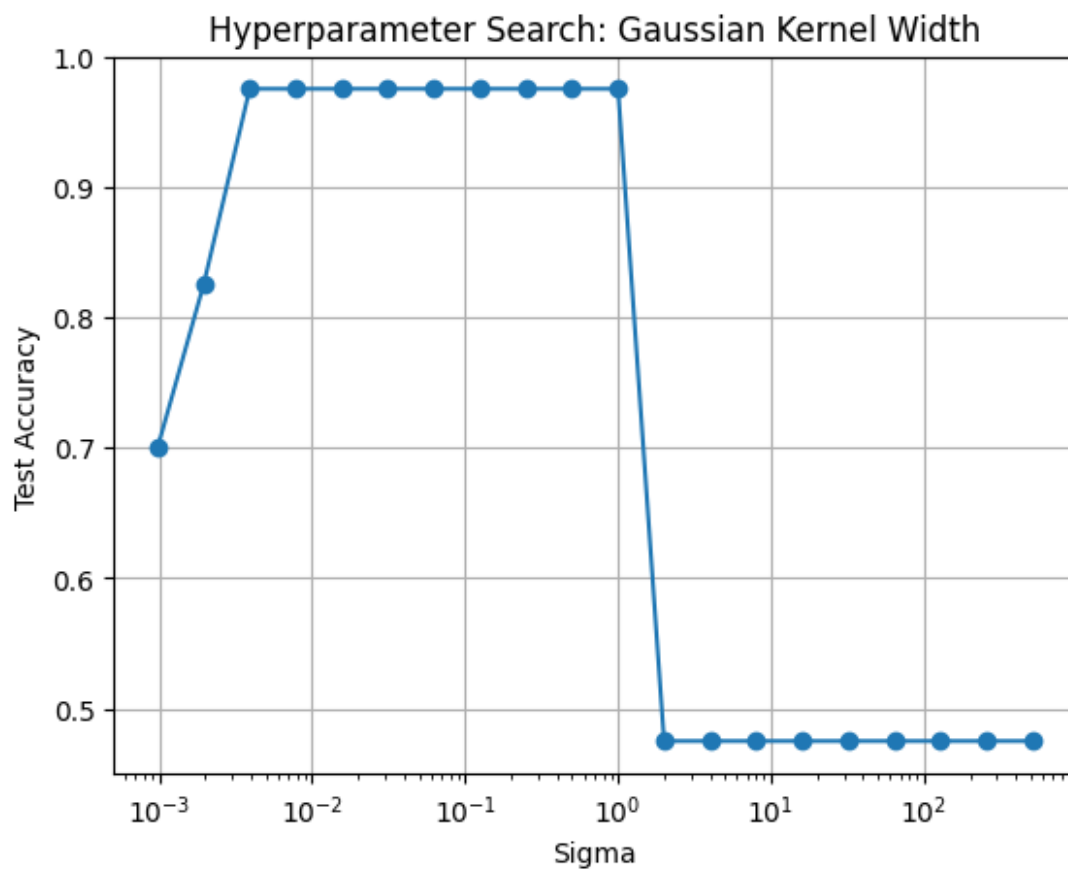
print("-" * 25)
print(f"Optimal Sigma: {best_sigma} with Accuracy: {best_accuracy:.2f}")

# Plot the relationship
plt.plot(sigmas, accuracies, marker='o')
plt.xlabel('Sigma')
plt.xscale('log')
plt.ylabel('Test Accuracy')
plt.title('Hyperparameter Search: Gaussian Kernel Width')
plt.grid(True)
plt.show()
```

Sigma	Accuracy

0.00098	0.7000
0.00195	0.8250
0.00391	0.9750
0.00781	0.9750
0.01562	0.9750
0.03125	0.9750
0.06250	0.9750
0.12500	0.9750
0.25000	0.9750
0.50000	0.9750
1.00000	0.9750
2.00000	0.4750
4.00000	0.4750
8.00000	0.4750
16.00000	0.4750
32.00000	0.4750
64.00000	0.4750
128.00000	0.4750
256.00000	0.4750
512.00000	0.4750

Optimal Sigma: 0.00390625 with Accuracy: 0.97



TODO: Describe your code

i create a KernelPerceptron instance using a GaussianKernel configured with the current sigma s. I fixed the number of epochs to 10. The model computes the Gram matrix and learns the support vector coefficients (α) using the training data. Then, the trained model predicts labels (ypred) for the unseen test set Xtest. The predictions are compared against the true labels ytest to calculate the accuracy percentage (acc), and the result is stored in the accuracies list to plot the "Sigma vs. Accuracy" curve later. Inside the loop, I check if the current model's accuracy is better than the previous best. If so, I update best_accuracy and record s as the new best_sigma. Finally, the code plots sigma vs. test accuracy.